Advanced Object Oriented Programming

Xingjie ZHANG

아주대학교
AJOU UNIVERSITY

아주대학교
AJOU UNIVERSITY

Class

Instructor: Xingjie ZHANG

Email : xzhang@ajou.ac.kr

Wechat : runningSnaile

CONTENTS

01 OOP

02 Class & Instance

03 Constructor

04 Access Control

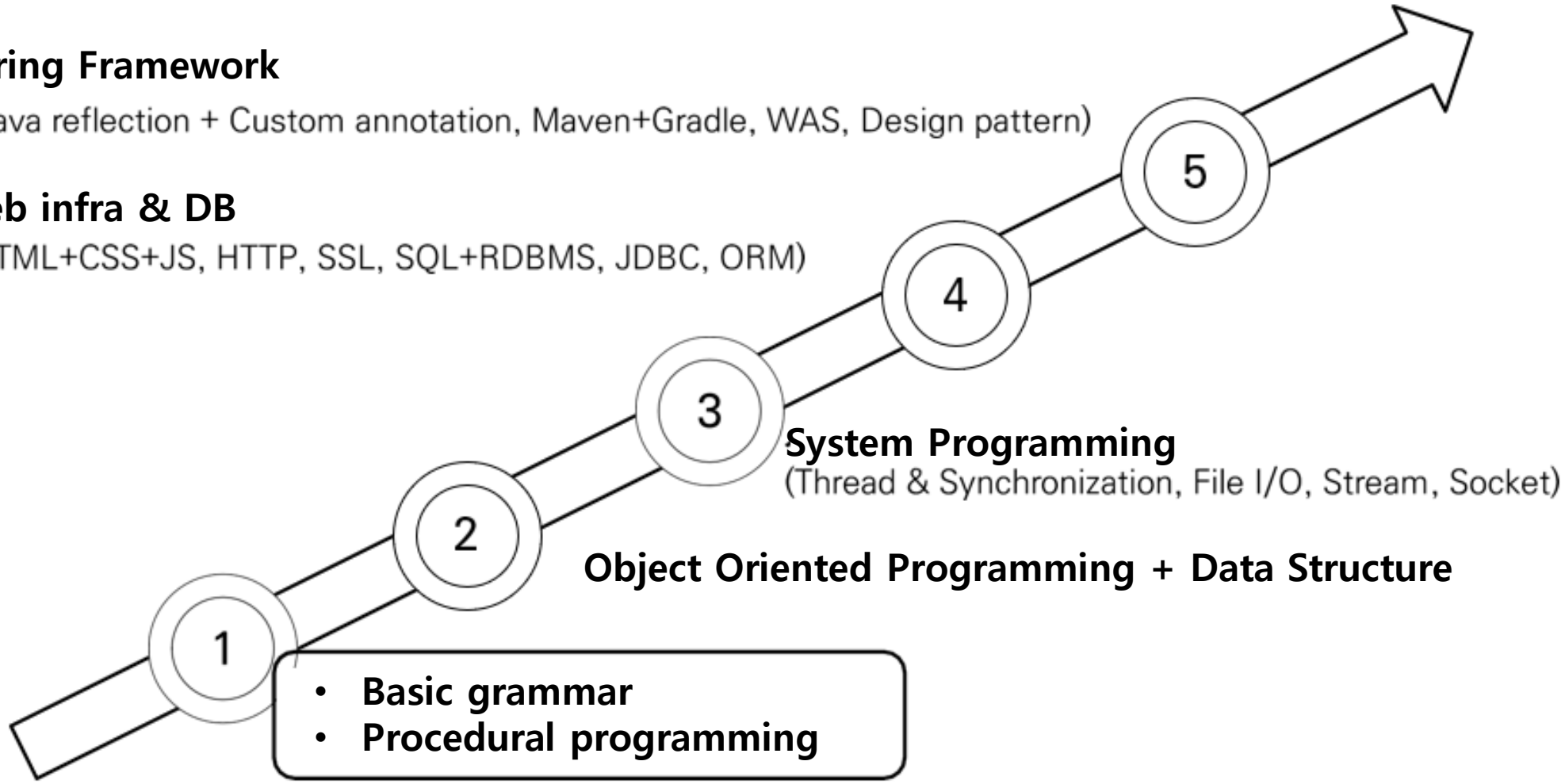
05 this

아주대학교
AJOU UNIVERSITY

01

OOP

Java Learning Order



Low dependence on specific languages

Before starting to study OOP

- Basic CS knowledge
- Procedural programming
- Basic programming language knowledge

Before starting to study OOP

- Differences from C++
  - Java has no destructors (replaced by **GC**).
  - Java lacks constant methods or constant references, and instead uses immutable objects to compensate.
  - This can be more efficient in architectures that utilize GC.
  - String constants are managed by String class objects, and the JVM manages constants.

Before starting to study OOP

- Differences from C++
  - In Java, void type variables cannot be declared and are only used as return types.
  - The null value can only be applied to reference types.
  - No operator functions.
  - In Java, all class types are references.

Before starting to study OOP

- Differences from C++
  - Relational operations on reference types compare object memory addresses.
  - Reference values can be changed.

02

Class & Instance

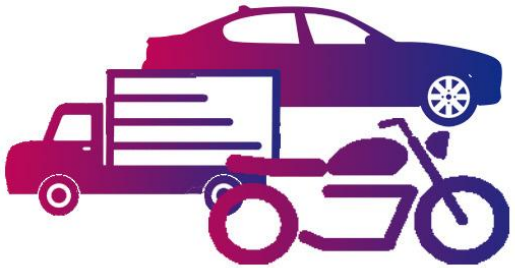
Object

- What is an object?
  - An object is the **basic unit** of source code in an OOP environment.
  - A collection of variables and their **associated** functions.
  - A **class** describes an object.

Class & Instance

blue print of an object

```
class Vehicle{
    int numberOfWheels;
    String brandName, color;
    double price;
    void start( );
    void changeGear( );
}
```



```
Vehicle car = new Vehicle( );
```

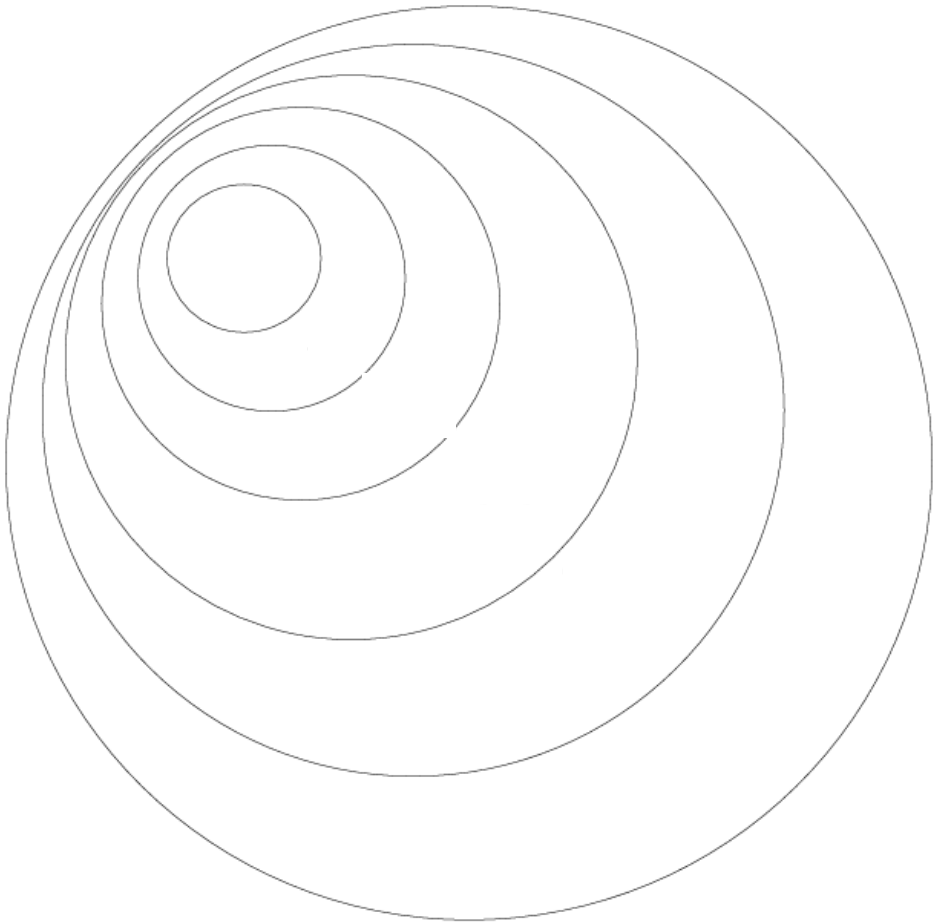
actual object

```
Vehicle bike = new Vehicle( );
```

```
Vehicle truck = new Vehicle( );
```

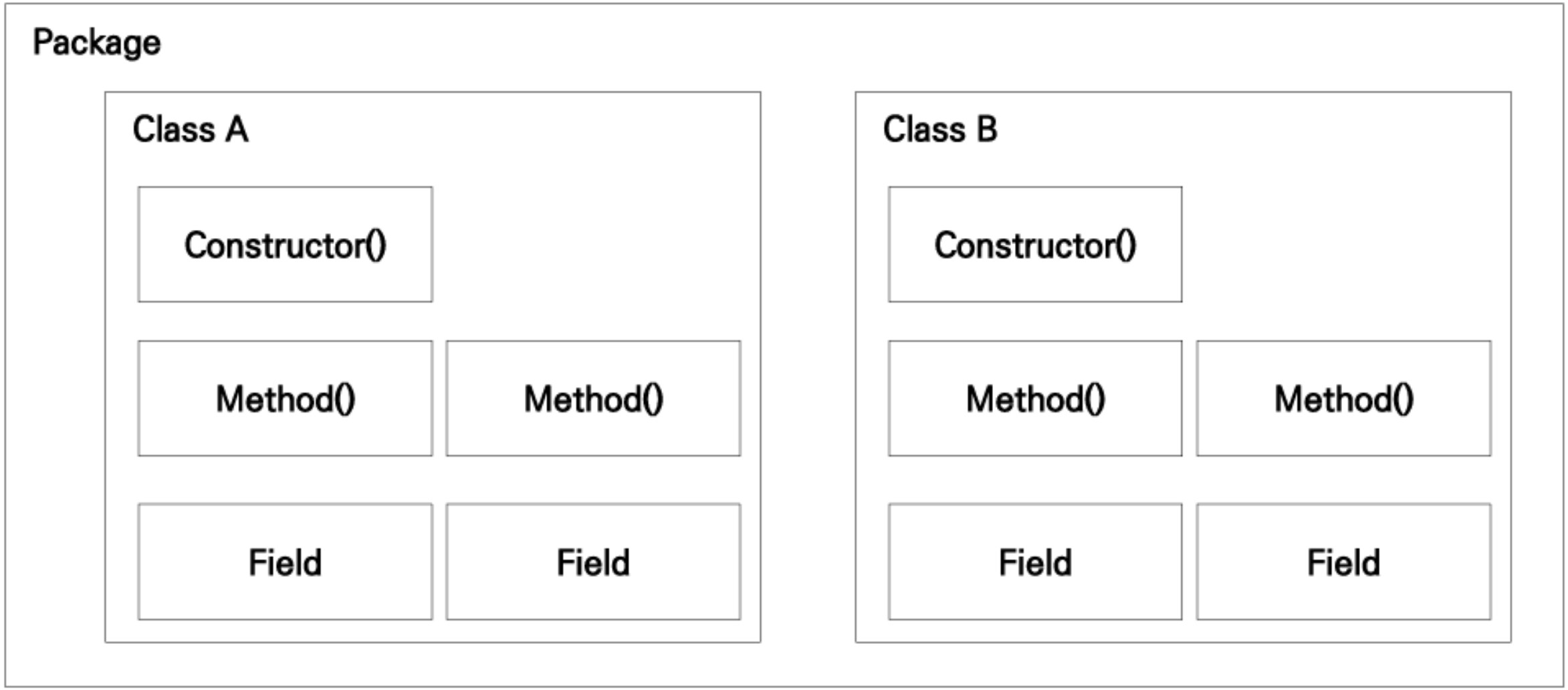
www.btechsmartclass.com

Class & Instance



Terms, expressions, statements, methods, classes, packages

Class & Instance



Object

- The general rule is to write class names in English, with the first letter **capitalized**.
- The class name and the **.java** file in which it is saved **must be the same**.
  - Required for **public declarations**.
  - If multiple classes are defined in a single file, separate files are not required.

```
class <ClassName>{
    data members declaration;
    methods defination;
}
```

Object

- Initial values can be declared when declaring a field. If not specified, the **default value is 0**.

- Initialization can be done via a constructor.
  - If the initial value specified in the field declaration and a constructor exist, the initialization specified in **the constructor takes precedence**.

Object

- _classSample

```
class MyFirstClass {
    private int data;

    public int getData() {
        return data;
    }

    public void setData(int param) {
        data = param;
    }
}
```

Object

- _classSample

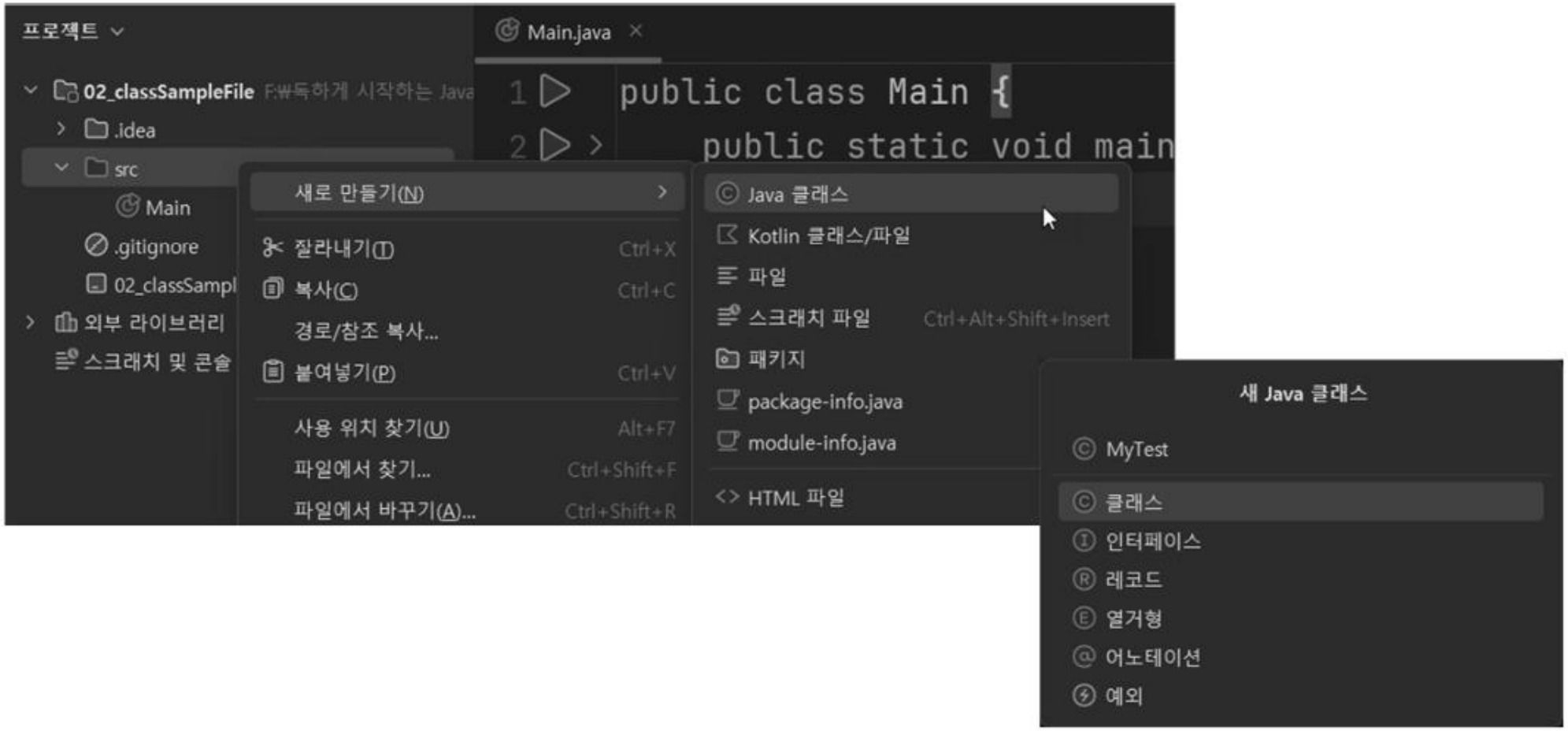
```
public class Main {
    public static void main(String[] args) {
        MyFirstClass myClass = new MyFirstClass();

        System.out.println(myClass.getClass());
        System.out.println(myClass.getData());
        //myClass.data = 10;
        myClass.setData(10);
        System.out.println(myClass.getData());
    }
}
```

```
class MyFirstClass
0
10
```

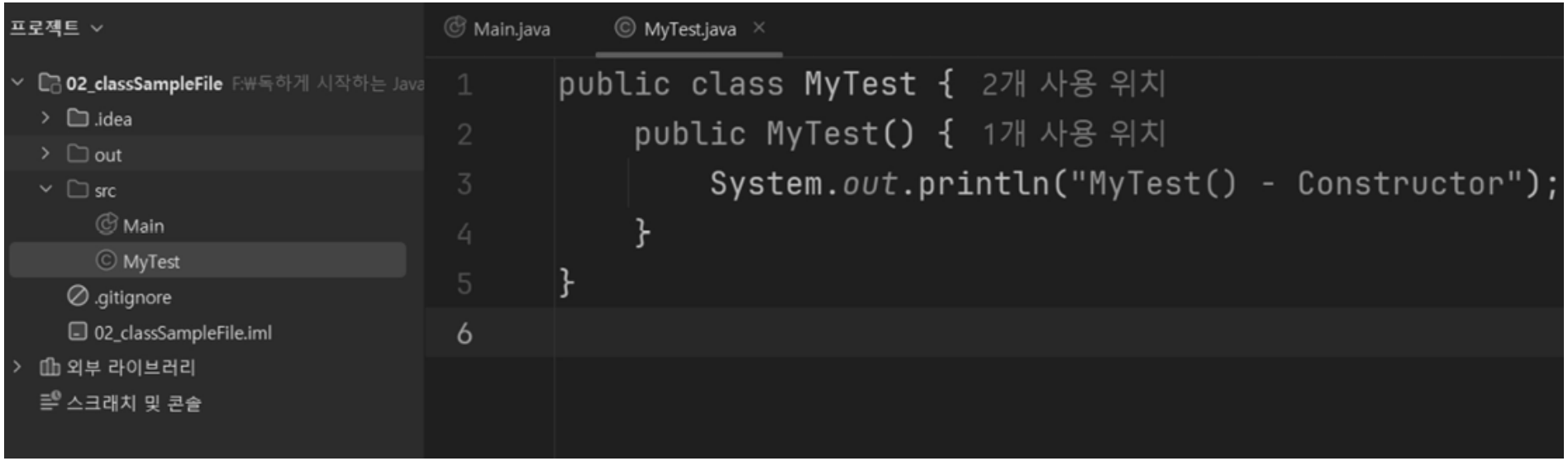
Object

- _classSampleFile



Object

- _classSampleFile




```
public class Main {
    public static void main(String[] args) {
        MyTest t = new MyTest();
        System.out.println("end of Main.main()");
    }
}
```

03

Constructor

Constructor

- A constructor is a method that is automatically called when an object is created.
  - It must have **the same name as the class**.
  - If omitted, the compiler automatically adds an empty constructor.
  - There are no return values or callers.
- Objects are **created dynamically** using the new operator.
- Multiple definitions are allowed.
- Only code related to the class itself should be described.

Constructor

- Dynamic creation of instances
  - Dynamically create instances using the new operator.
  - All variable declarations for class types are references.
  - If an instance does not exist, null is returned.
  - An instance can have multiple references.

- Side-effect issues

Object

- _constructor

```
class MyTest {
    private int data = 5;

    MyTest() {
        data = 10;
    }

    public int getData() {
        return data;
    }
}

public class Main {
    public static void main(String[] args) {
        MyTest t = new MyTest();
        System.out.println(t.getData());
    }
}
```

Constructor

- Dynamic creation of instances
  - Dynamically create instances using the new operator.
  - All variable declarations for class types are references.
  - If an instance does not exist, null is returned.
  - An instance can have multiple references.

- Side-effect issues

04

Access Control

Access Control

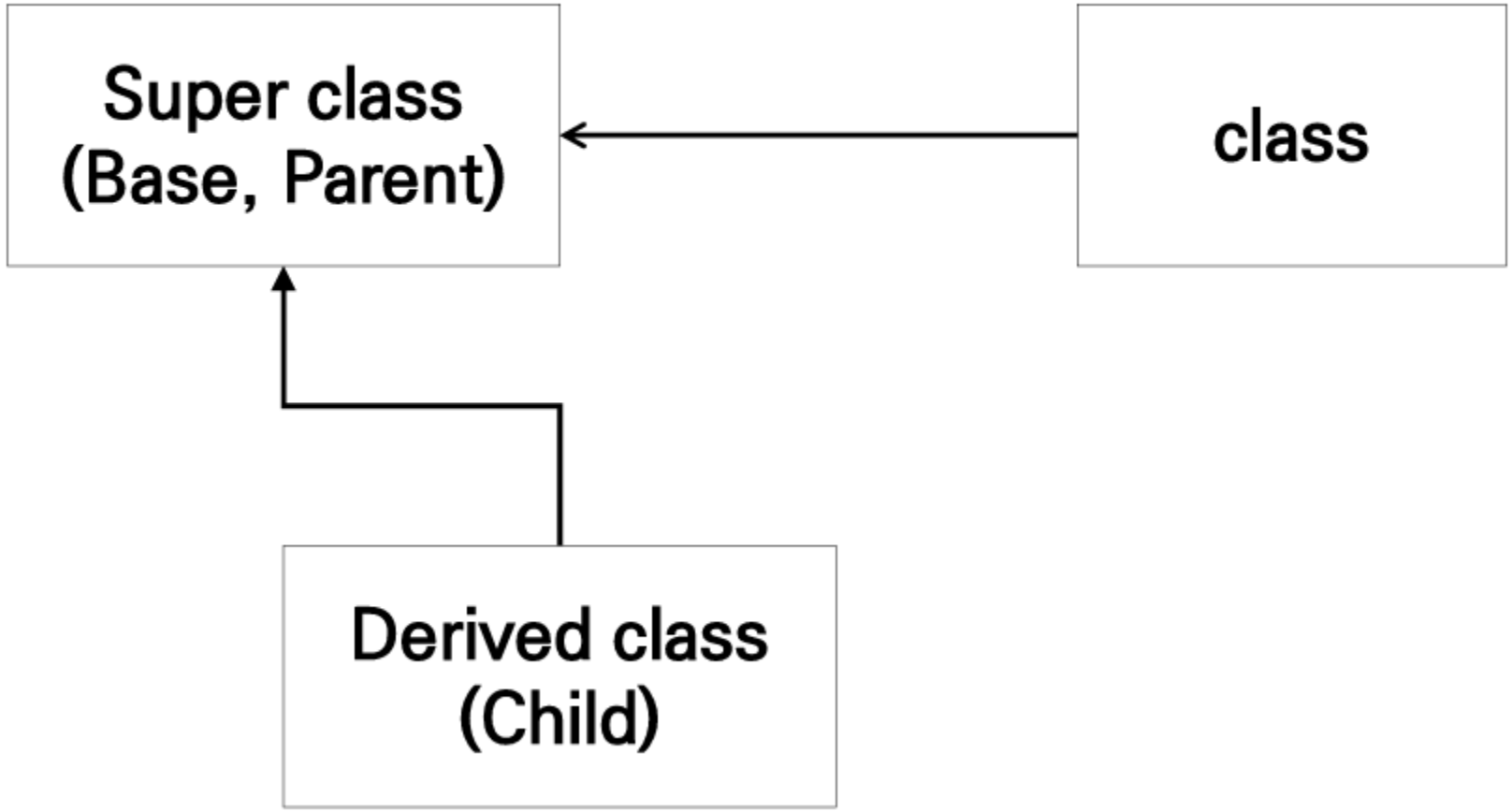
- The **public** members can be accessed everywhere.
- The **private** members can be accessed only inside the same class.
- The **protected** members are accessible to every child class (same package or other packages).
- The **default** members are accessible within the same package but not outside the package.

Access Control

Access control for members of class and interface in java

Accessibility Location / Access Specifier	Same Class	Same Package		Other Package	
		Child class	Non-child class	Child class	Non-child class
Public	Yes	Yes	Yes	Yes	Yes
Protected	Yes	Yes	Yes	Yes	No
Default	Yes	Yes	Yes	No	No
Private	Yes	No	No	No	No

Access Control



Access Control

Google Project Ara

VS

Access Control

- **Getter function** and **Setter function**
  - Methods for making hidden fields accessible from outside.
  - **Separate read and write** permissions for fields can be controlled.
  - Supports features such as automatic creation using tools.

- In object-oriented programming, the perspectives of creating objects and those of creating and using them must be separated.

05

“this” Referrer

“this” Referrer

- A reference to **an object's own instance**.
- The value is determined when an instance of the class is declared.
- The value is stored on the stack (slot 0) when a method is called.
- Cannot be used in **static methods**.

“this” Referrer

- _thisSample

```
class MyTest {
    public int data;

    MyTest(int param) {
        this.data = param;
    }

    void printData() {
        int data = 3;
        System.out.println(this.data + data);
    }
}
```

“this” Referrer

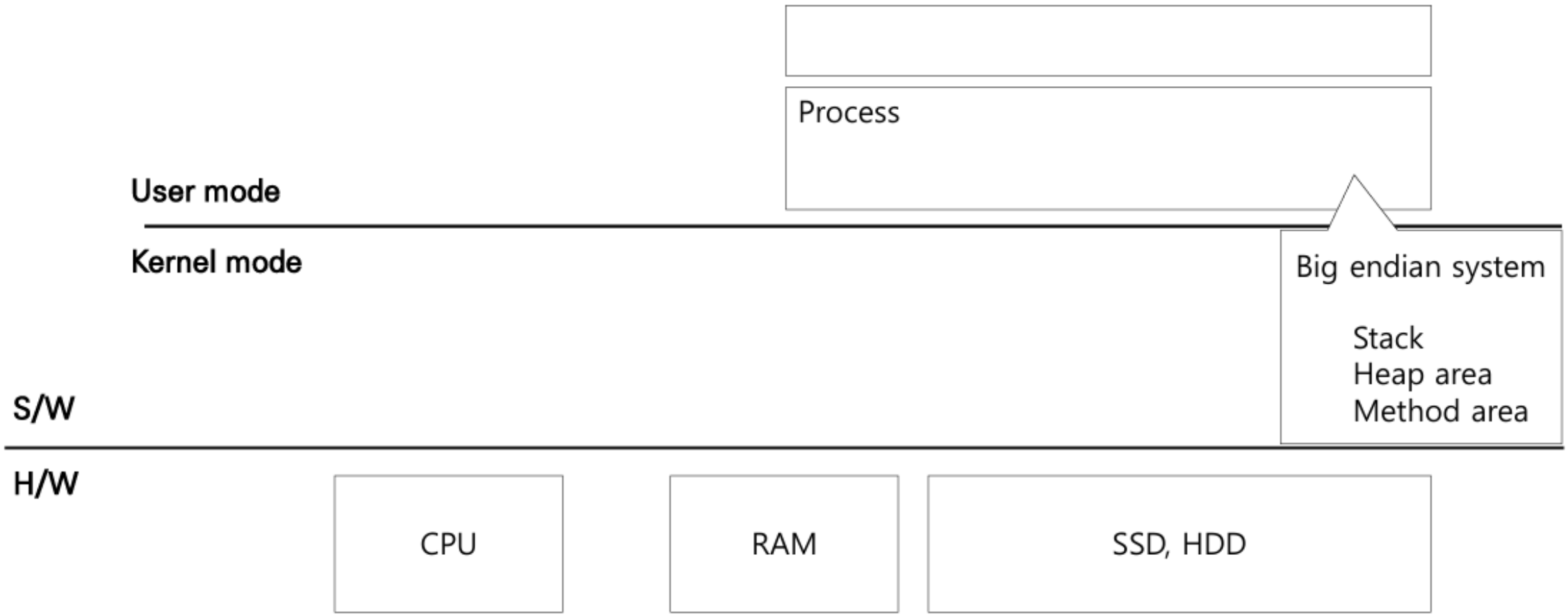
- _thisSample

```
public class Main {
    public static void main(String[] args) {
        MyTest t1 = new MyTest(5);
        MyTest t2 = new MyTest(10);

        System.out.println(t1.data);
        System.out.println(t2.data);
        t1.printData();
    }
}
```

```
5
10
8
```

“this” Referrer



“this” Referrer

- _thisAndStatic

```
class MyTest {
    private int data;
    public int getData() {
        return data;
    }
    public void setData(int param) {
        data = param;
    }
    public static void setStatic(int param) {
        this.data = param;
    }
}
```

```
F:\독하게 시작하는 Java - Part 2\02_thisAndStatic\src\Main.java:10:9
java: non-static variable this cannot be referenced from a static context
```

“this” Referrer

- _thisAndStatic

```
class MyTest {
    private int data;
    public int getData() {
        return data;
    }
    public void setData(int param) {
        data = param;
    }
    public static void setStatic(MyTest data, int param) {
        //this.data = param;
        data.data = param;
    }
}
```

Quiz

- 1. Which of the following best describes the relationship between a 'class' and an 'instance' in Java?
  A. A class contains actual data, and an instance serves as a blueprint.
  B. A class serves as a blueprint for objects, and an instance is an actual object created in memory based on that blueprint.
  C. Class and instance are exactly the same concept, only with different names.
  D. An instance refers to the functions defined inside a class.

Quiz

- 2. Which of the following correctly lists the essential components typically included in a Java class?
  A. It includes only methods.
  B. It includes fields and methods, and can also have constructors.
  C. It includes only instances and references.
  D. It includes only packages and libraries.

Quiz

- 3. What is the main role of a constructor in Java?
  A. It is a function that performs general operations within a class.
  B. It is automatically called when an instance of the class is created and is responsible for initializing the object.
  C. It releases memory when an object is no longer in use.
  D. It controls access to the class's members from other classes.

Quiz

- 4. What is the main reason for declaring a class field as private?
  A. To allow access to the field only from child classes that inherit it.
  B. To restrict direct access to the field to within the class itself, thereby protecting data integrity by preventing external modification.
  C. To increase the program's execution speed.
  D. To prevent the field from being allocated in memory.

Quiz

- 5. What does the this keyword refer to in Java?
  A. It refers to the class itself and is used to access class members.
  B. It refers to another object that called the currently executing method.
  C. It is a reference to the object that called the method, i.e., the current instance being worked on.
  D. It refers to static members shared by all instances of the class.

Practice

- 按照如下要求实现一个简化版的String类(MyString)
  - Field
    - byte[] string
  - Method
    - string getString()
    - void setString(String param)
    - int length()
    - int compareTo(String param)
  - 包含如上功能且不限制其他功能
  - 需要设计一个测试代码来验证

```
public class Main {
    public static void main(String[] args) {
        MyString str = new MyString();
        System.out.println(str.length());
        System.out.println(str.getString());

        str.setString("Hello Java");
        System.out.println(str.getString());
        System.out.println(str.length());
        System.out.println(str.compareTo("Hello Java"));
    }
}
```

Q & A

THANK YOU